

Unit 1 Collaborative Discussion – Summary Post

My initial post examined OWASP A03: Injection, arguing that SQL injection vulnerabilities arise from cumulative design and review failures rather than isolated coding mistakes. As Olmsted (2024) emphasises, secure development must embed preventative controls from the design phase rather than rely on post-deployment detection. By modelling the vulnerability through a UML activity diagram, I illustrated how insecure query construction, absent validation, and lack of security-focused code review collectively enable exploitability. This supports the view that weaknesses often reflect systemic process deficiencies rather than individual developer error.

Peer discussions refined this analysis in several important ways. Zahid's suggestion to incorporate SAST and DAST prompted clarification regarding their placement within the Secure SDLC. While these tools operate beyond a single vulnerability model, acknowledging lifecycle-wide controls such as SonarQube for static analysis and Burp Suite for dynamic testing strengthens defence-in-depth. For example, parameterised queries prevent injection at code level, while static analysis tools can detect unsafe query construction before deployment.

In responding to Tahmeed, I recognised the importance of scope precision within OWASP A03. Injection encompasses multiple interpreter-based attacks (OWASP, 2021), and UML artefacts must clearly specify whether they represent SQL injection specifically or injection as a broader class. Clear categorisation improves mitigation accuracy.

The discussion with Moses reinforced the importance of distinguishing root causes from impact-amplifying control failures. Evidence indicates that the Equifax breach originated from an unpatched Apache Struts vulnerability rather than cryptographic design failure (Kabanov and Madnick, 2020). This distinction strengthened my analytical discipline when classifying vulnerabilities under OWASP categories.

In future professional practice, I will apply this causal analysis approach when assessing vulnerabilities, distinguishing root causes from amplifying factors to enable faster and more precise risk mitigation.

References

Kabanov, I. and Madnick, S. (2020) *A systematic study of the control failures in the Equifax cybersecurity incident*. MIT Sloan Research Paper No. 2020–19. Available at: <https://web.mit.edu/smadnick/www/wp/2020-19.pdf>

NIST (2022) *Secure Software Development Framework (SSDF), SP 800-218*. Available at: <https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-218.pdf>

Olmsted, A. (2024) *Security-Driven Software Development*. Birmingham: Packt Publishing. Available at: <https://www.packtpub.com/en-sg/product/security-driven-software-development-9781835461174>

OWASP (2021) *OWASP Top 10: A03 – Injection*. Available at: https://owasp.org/Top10/A03_2021-Injection/